Unsecure vulnerability

SQLi

```php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $username = $_POST['username'];
    $password = $_POST['password'];

    // VULNERABILITY: SQL Injection (SQLi)
    $query = "SELECT * FROM users WHERE username = '$username' AND password = '$password'";
    $result = mysqli_query($conn, $query);

    if ($result && mysqli_num_rows($result) > 0) {
        $user = mysqli_fetch_assoc($result);
```

Aplikasi mengambil input $username dan $password dari pengguna lalu menyambungkannya secara langsung ke dalam query SQL tanpa difilter.

BAC

```php
$note_id = $_GET['id'];

$query = "SELECT * FROM notes WHERE id = '$note_id'";
$result = mysqli_query($conn, $query);
```

Aplikasi mengambil parameter id dari URL lalu langsung mencari data di database tanpa mengecek apakah ID pengguna yang sedang aktif adalah pemilik catatan tersebut.

Sensitive Data *Exposure*

```sql
INSERT INTO users (username, password, is_admin) VALUES
('admin', 'admin123', 1),
('rahasia_admin', 'supersemar99', 1),
('johndoe', 'password123', 0),
('janedoe', 'rahasia123', 0);
```

Kata sandi (password) pengguna disimpan dalam bentuk teks terbuka(plaintext) di database jika peretas berhasil mendapat akses ke database, mereka dapat melihat kata sandi asli milik semua pengguna.

Secure

SQLi

```
$stmt = mysqli_prepare($conn, "SELECT * FROM users WHERE username = ?");
mysqli_stmt_bind_param($stmt, "s", $username);
mysqli_stmt_execute($stmt);
$result = mysqli_stmt_get_result($stmt);

if ($result && mysqli_num_rows($result) > 0) {
    $user = mysqli_fetch_assoc($result);
```

Parameter $username sekarang dimasukkan melalui Prepared Statement menggunakan fungsi mysqli_prepare dan mysqli_stmt_bind_param. Peretas tidak bisa menyuntikkan ' OR 1=1 -- -.

Broken Access Control (BAC)

```
$note_id = $_GET['id'];

$stmt = mysqli_prepare($conn, "SELECT * FROM notes WHERE id = ? AND user_id = ?");
mysqli_stmt_bind_param($stmt, "ii", $note_id, $_SESSION['user_id']);
mysqli_stmt_execute($stmt);
$result = mysqli_stmt_get_result($stmt);

$row = mysqli_fetch_assoc($result);
```

Parameter ID (?id=1) divalidasi dengan Session user_id. Query hanya akan mengembalikan data jika ID Catatan tersebut milik User yang sedang Login.

Cross-Site Scripting (XSS)

```
    <h3><?php echo htmlspecialchars($row['title'], ENT_QUOTES, 'UTF-8'); ?></h3>
    <p>
        <em>Dibuat oleh User ID: <?= htmlspecialchars($row['user_id']) ?></em>
    </p>
    <div>
        <?php

        echo nl2br(htmlspecialchars($row['content'], ENT_QUOTES, 'UTF-8'));
        ?>
```

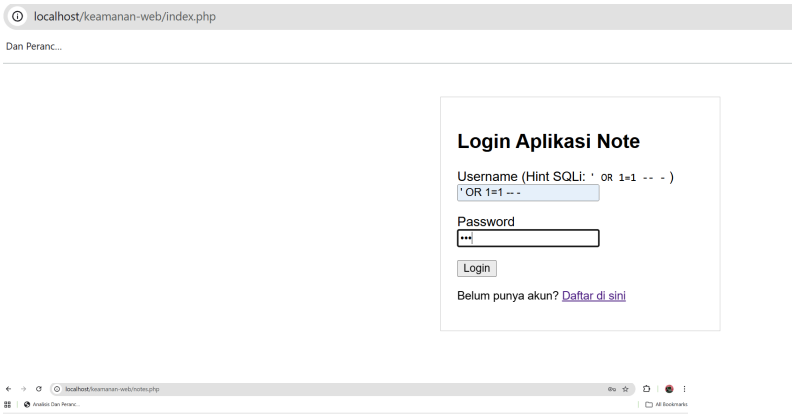
Menggunakan fungsi htmlspecialchars() saat menampilkan konten teks ke browser, mencegah eksekusi script seperti <script>alert('xss')</script>

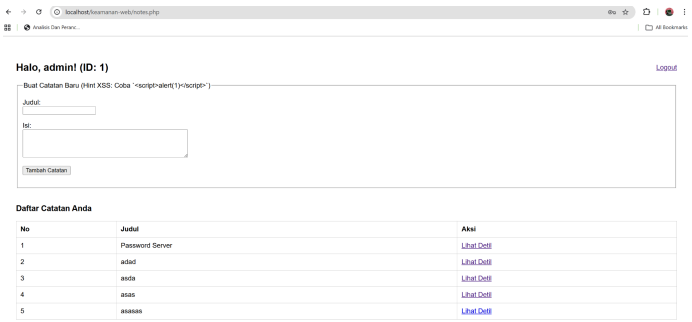
Sensitive Data Exposure

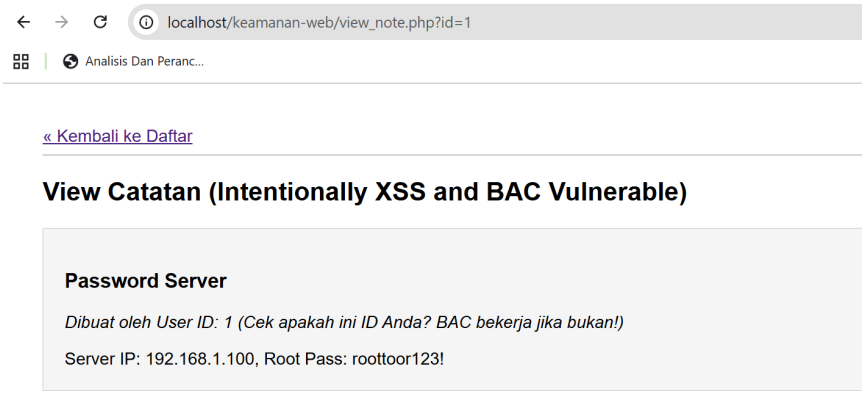
```sql
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL,
    password VARCHAR(255) NOT NULL,
    is_admin BOOLEAN DEFAULT FALSE
);

-- Tabel Notes
CREATE TABLE notes (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT,
    title VARCHAR(100),
    content TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

Plaintext dihilangkan kita ganti menggunakan hash

Unsecure web test





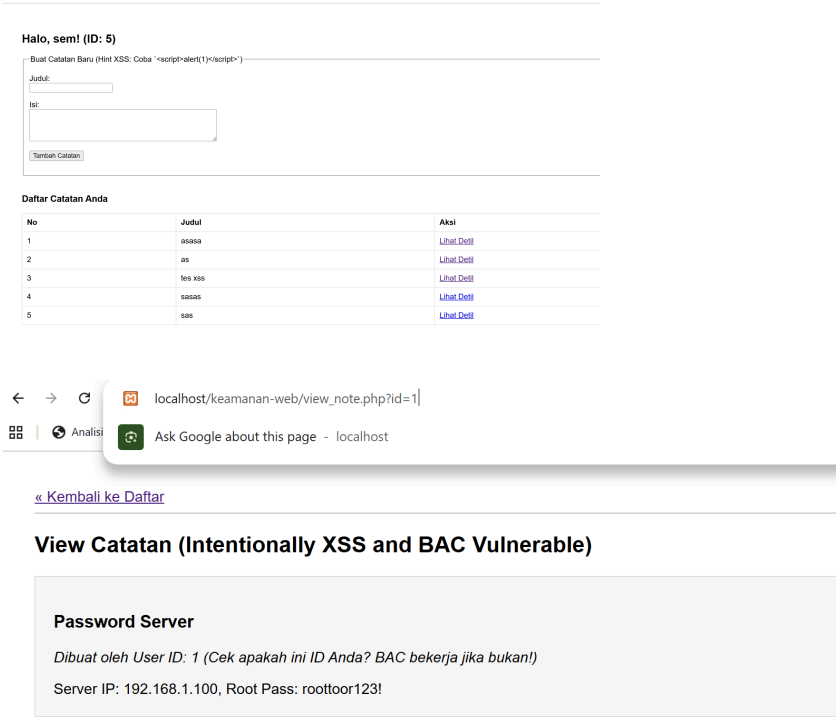


SQLi

Pengguna yang menggunakan Payload untuk login bisa masuk ke page admin dan dapat melihat dan edit semua isi dari postnya.

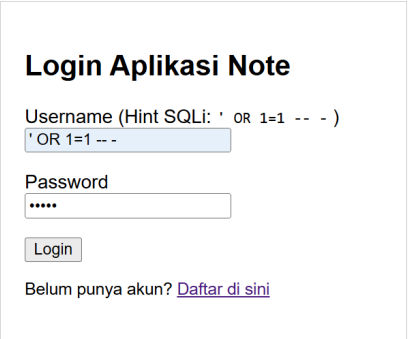
XSS

Pengguna dapat mengisi sintaks JavaScript/HTML berbahaya (seperti <script>), browser akan mengeksekusi .



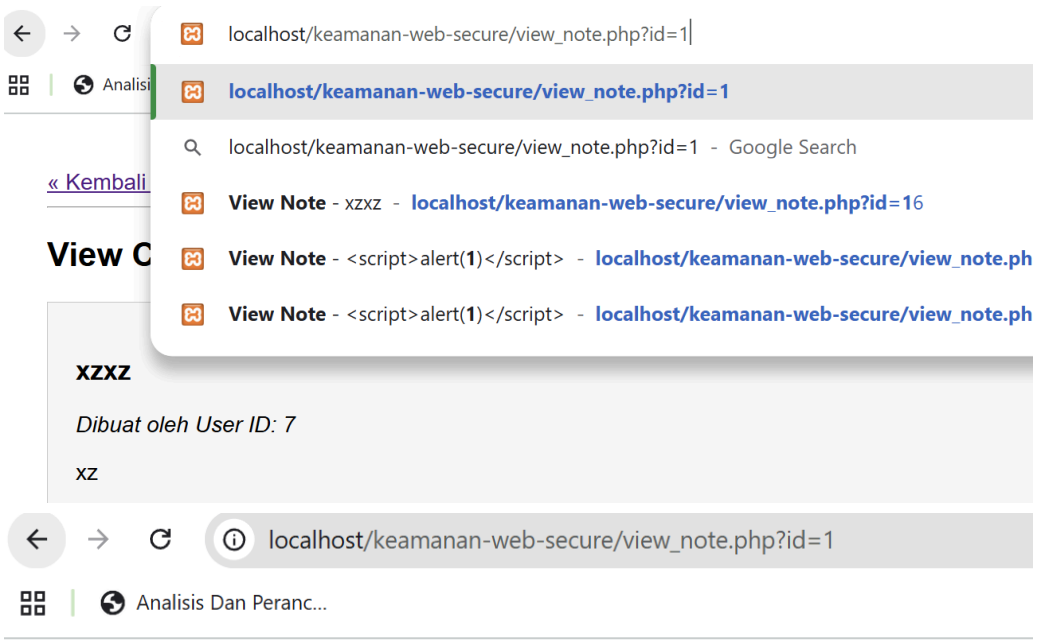
Dengan mengubah parameter ID di URL pengguna dapat masuk ke page yang seharusnya tidak bisa dia akses.

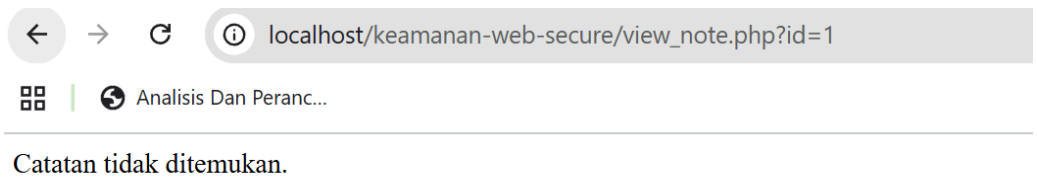
Secure web test



Penggunaan Payload terblokir

tidak memunculkan pop up artinya textnya hanya terbaca sebagai teks biasa.





Sudah tidak bisa mengakses note dengan mengubah Parameter ID di URL